



SCHOOL HOMEWORK - 2024

CLASS: CLOUD COMPUTING

SUBJECT: RESOURCES DEPLOYMENT THROUGH A CI/CD PIPELINE WITH
GIT, TERRAFORM AND AZURE

FRÉDÉRIC PERRON



Table of Contents

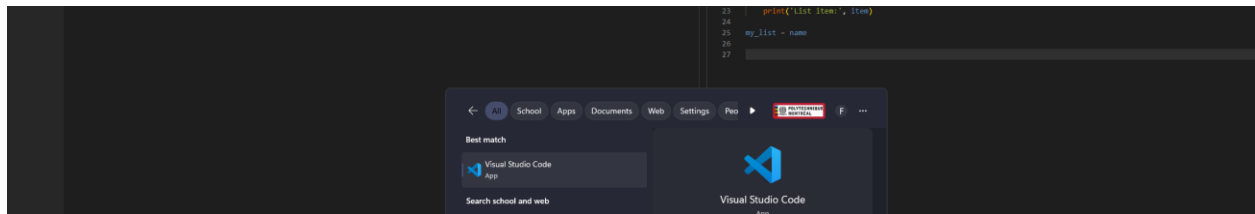
Introduction	1
Part 1 Installation & configuration of VS code.....	1
Part 2 Installation & configuration of GitHub Desktop.....	3
Part 3 Installation & configuration of Terraform CLI and Terraform cloud	5
Part 4 Configuration of Microsoft Azure account.....	13
Part 5 Integration and connection between Github and Terraform	14
Part 6 Integration and connection between Terraform cloud & MS Azure	15
Part 7 Deployment of group resources from your pipeline in MS Azure	18
Part 8 Deployment of a virtual network from your pipeline in MS Azure	19
Part 9 Deployment of a VM from your pipeline in MS Azure	20
Part 10 Deployment of a Docker container from your pipeline in MS Azure	22
Conclusion.....	26
References	27

Introduction

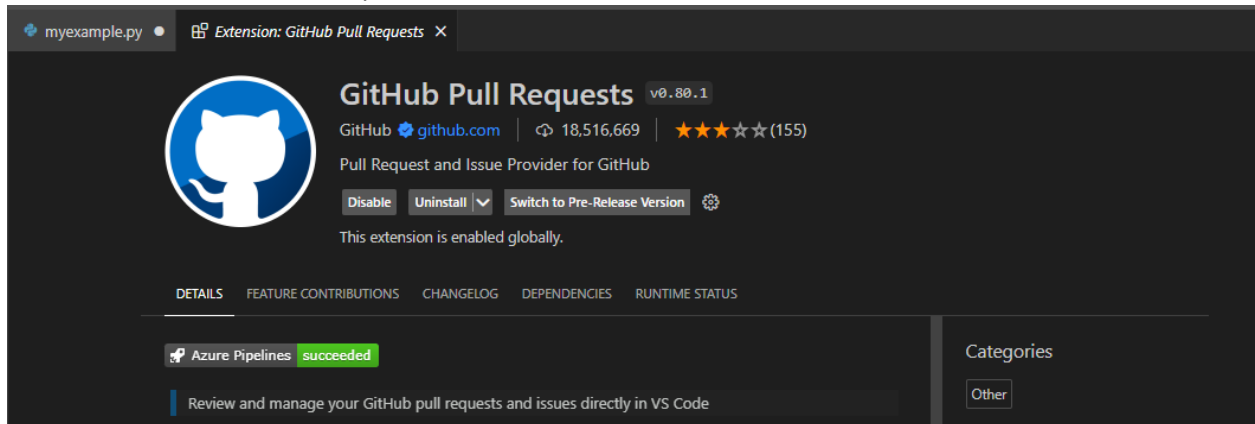
As part of this project, we carried out a series of steps to configure a development and deployment environment using several essential tools for managing infrastructure and cloud resources. Our main objective was to establish an efficient workflow for deploying infrastructure in Microsoft Azure, using tools such as Visual Studio Code, GitHub, Terraform CLI, Terraform Cloud, and Azure itself. This project allowed us to understand the steps required to set up a continuous deployment pipeline in the cloud.

Part 1 Installation & configuration of VS code

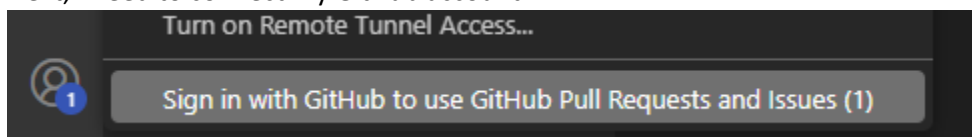
I already had Visual Studio Code installed, so I only needed to add the GitHub Pull Requests and Issues extension.

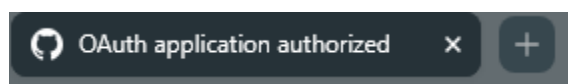
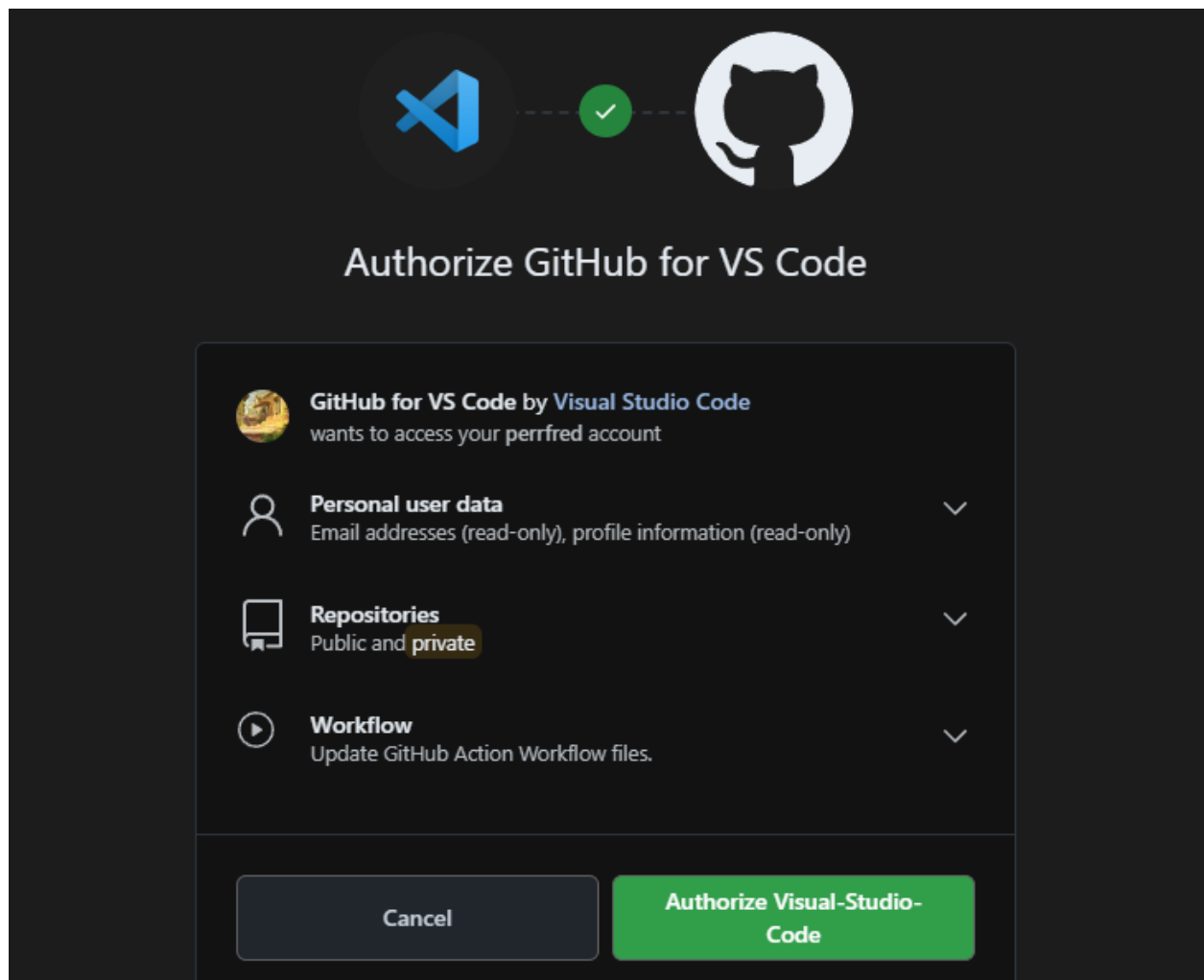


Now I add the GitHub Pull Requests extension:

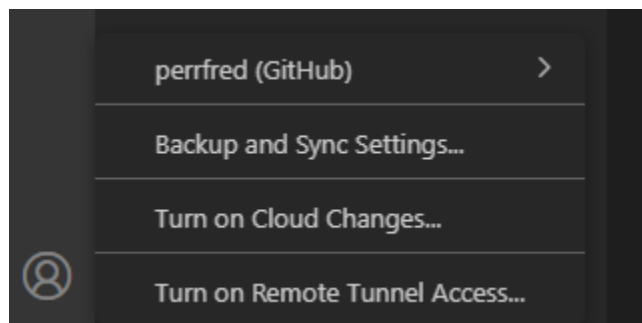


Next, I need to connect my GitHub account.





We can now see my personal GitHub account connected in Visual Studio Code.

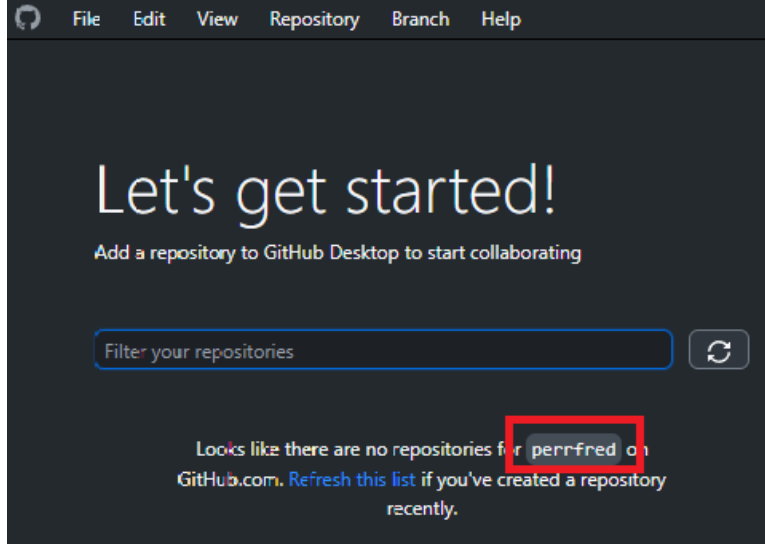


Part 2 Installation & configuration of GitHub Desktop

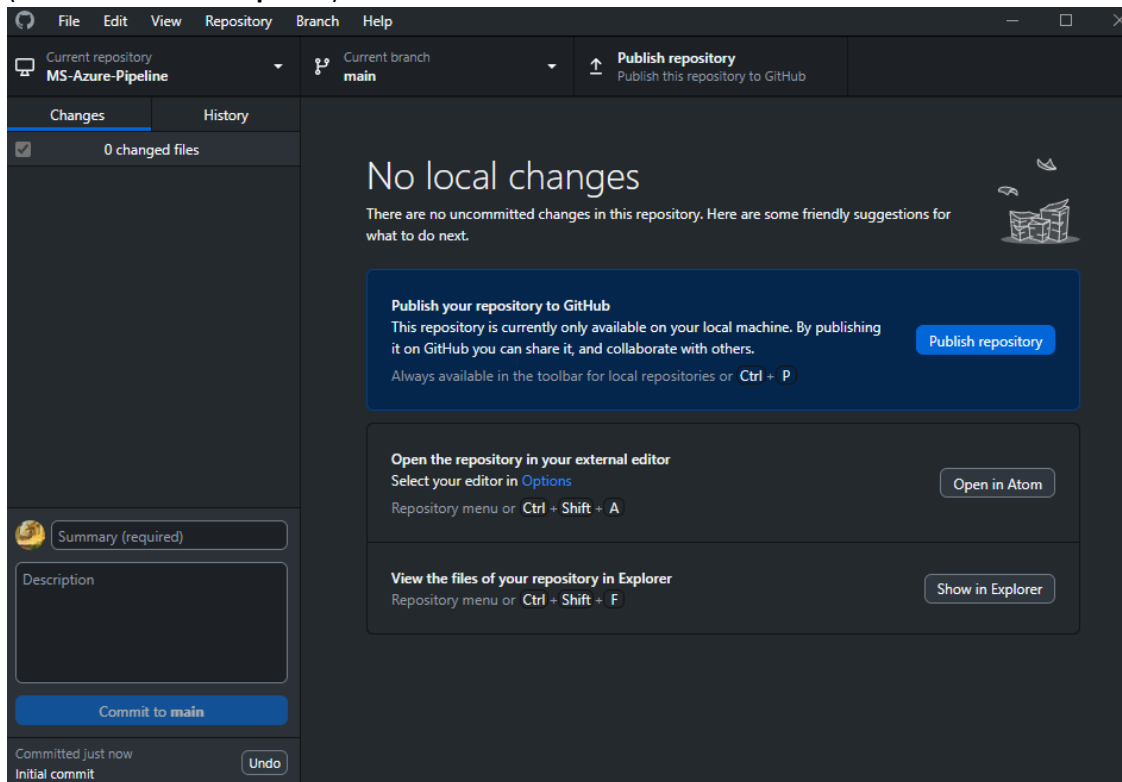
I started by installing GitHub Desktop from the official website.



Then, I linked my personal GitHub account (**perrfred**) to the GitHub Desktop application.

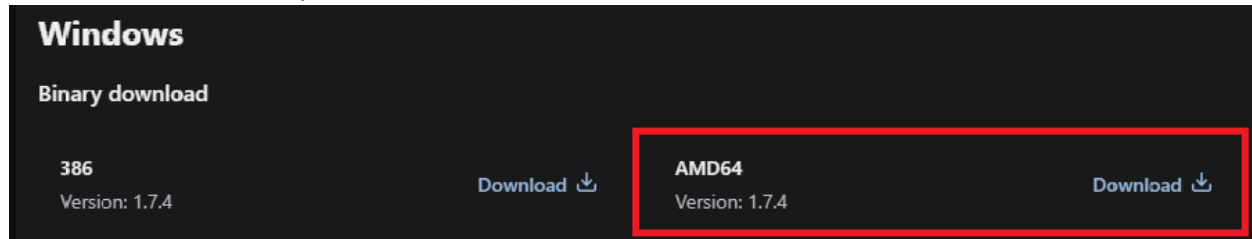


We then create a new repository for the course, which appears on the main page of GitHub Desktop (named **MS-Azure-Pipeline**).

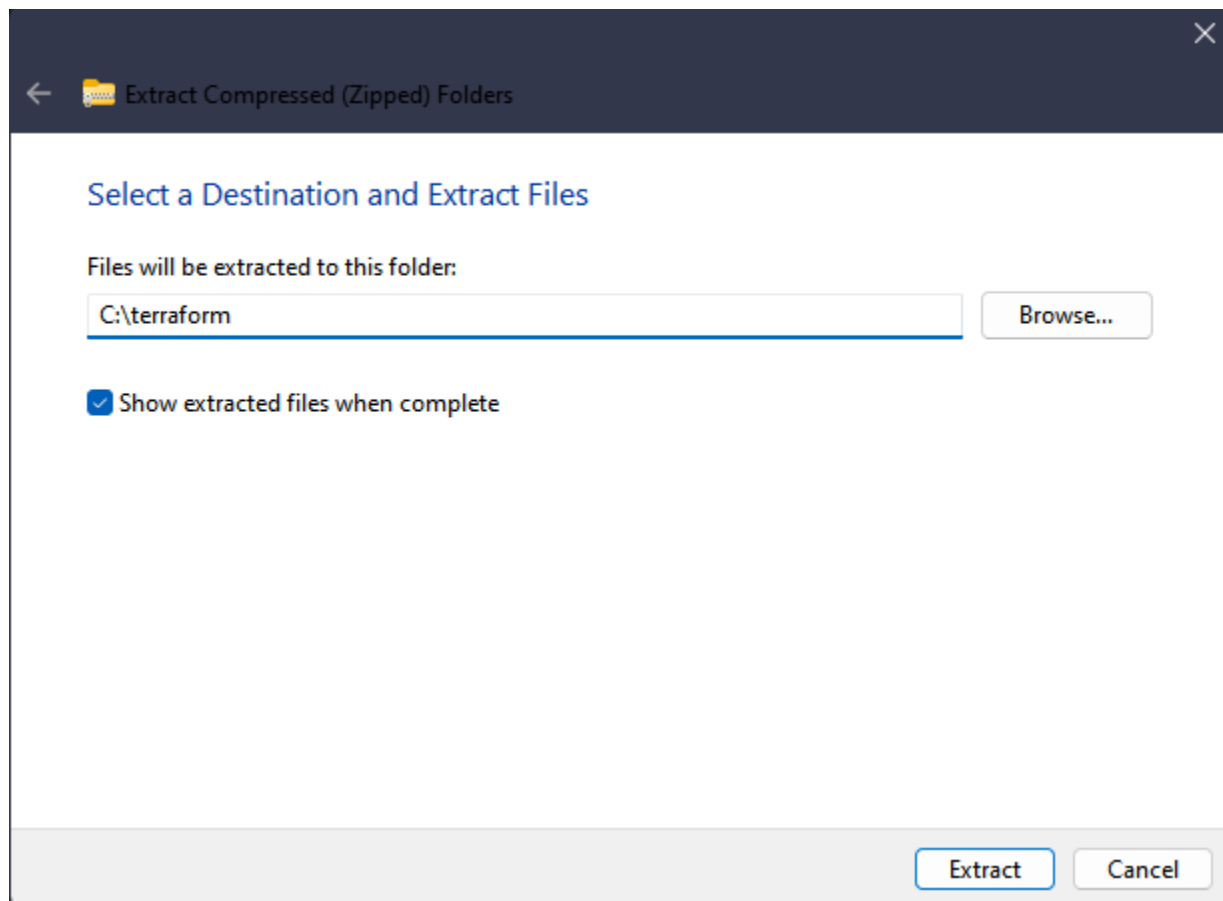
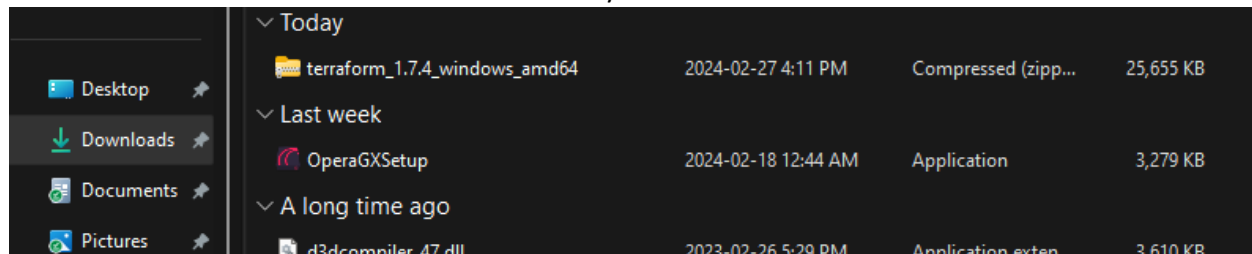


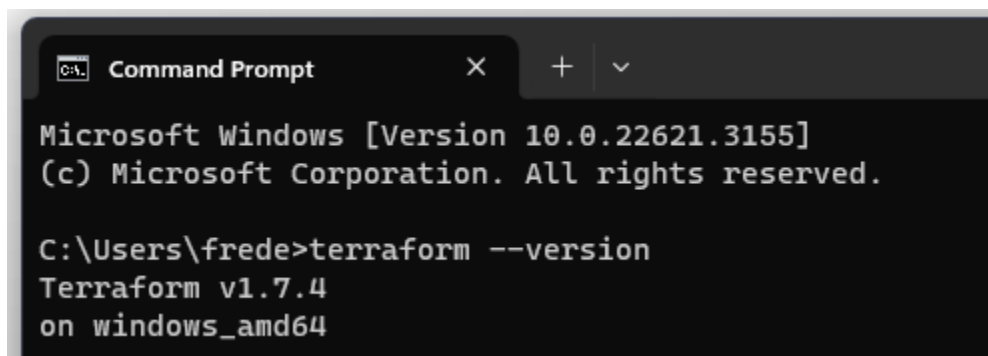
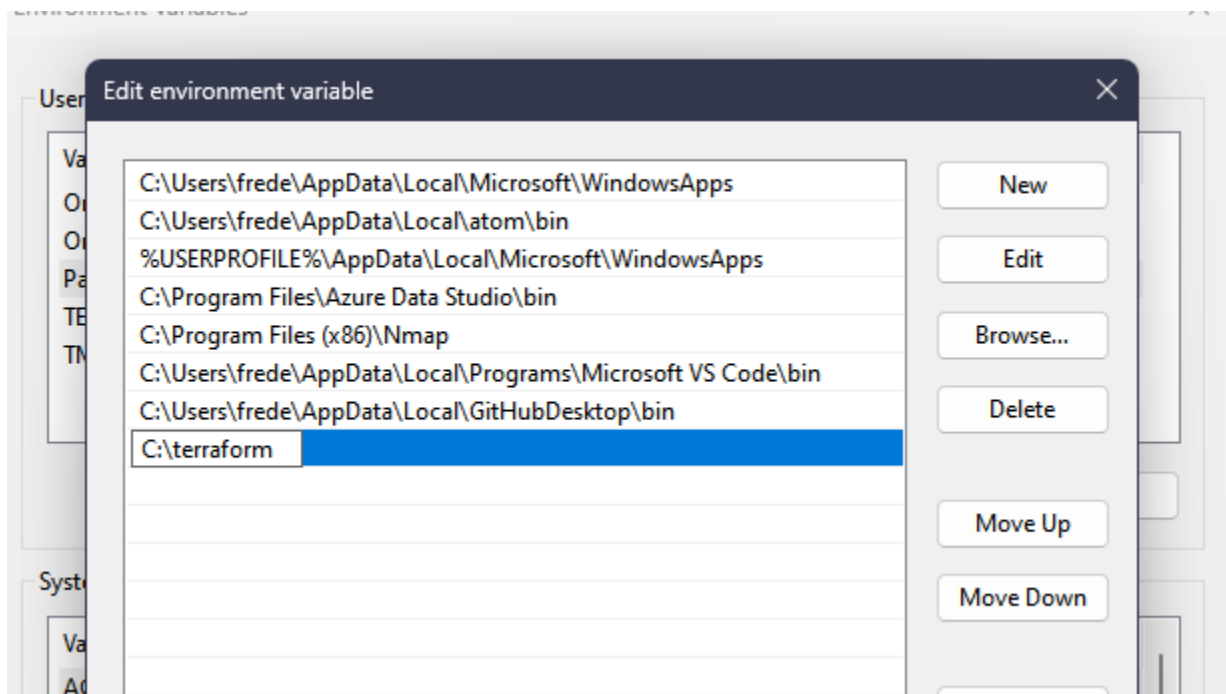
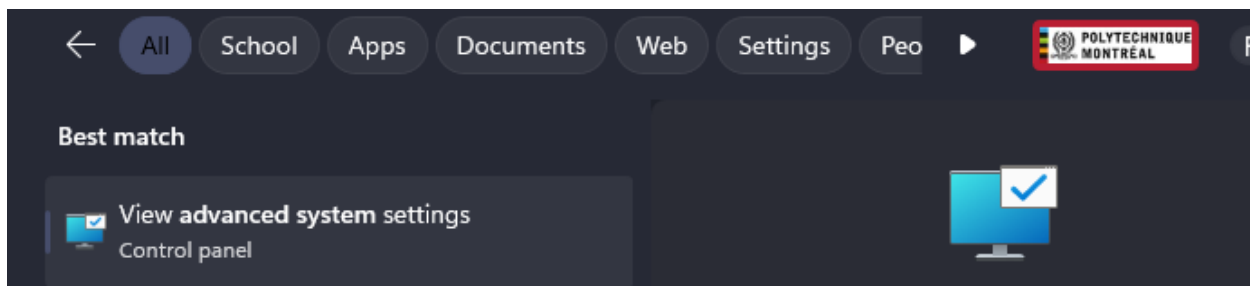
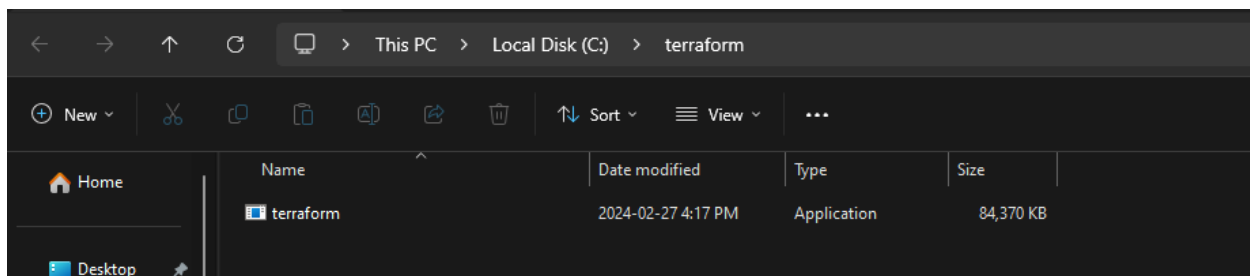
Part 3 Installation & configuration of Terraform CLI and Terraform cloud

Because I have a 64 bits processor (x86-64), I must download the version AMD64



We can then see the new folder downloaded in my Downloads





Create an account

Have an account? [Sign in](#)

 **Continue with HCP account**

OR

Username



Email



Password

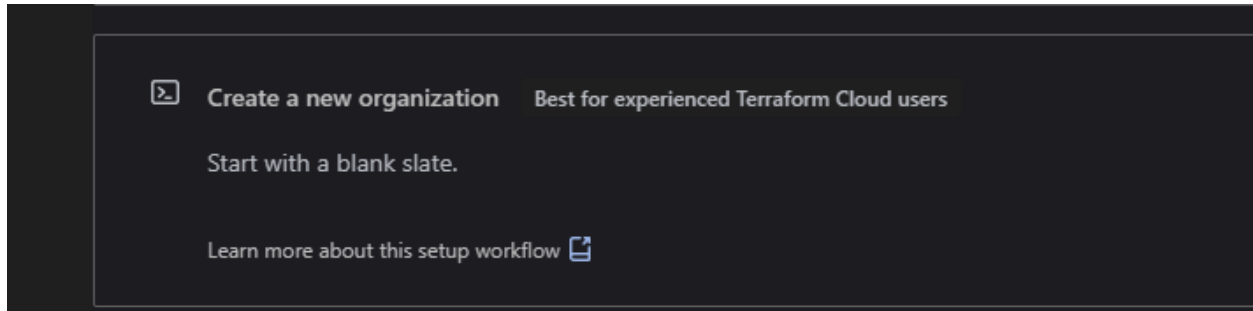
☒ I agree to the Terms of Use.

☒ I acknowledge the Privacy Policy.

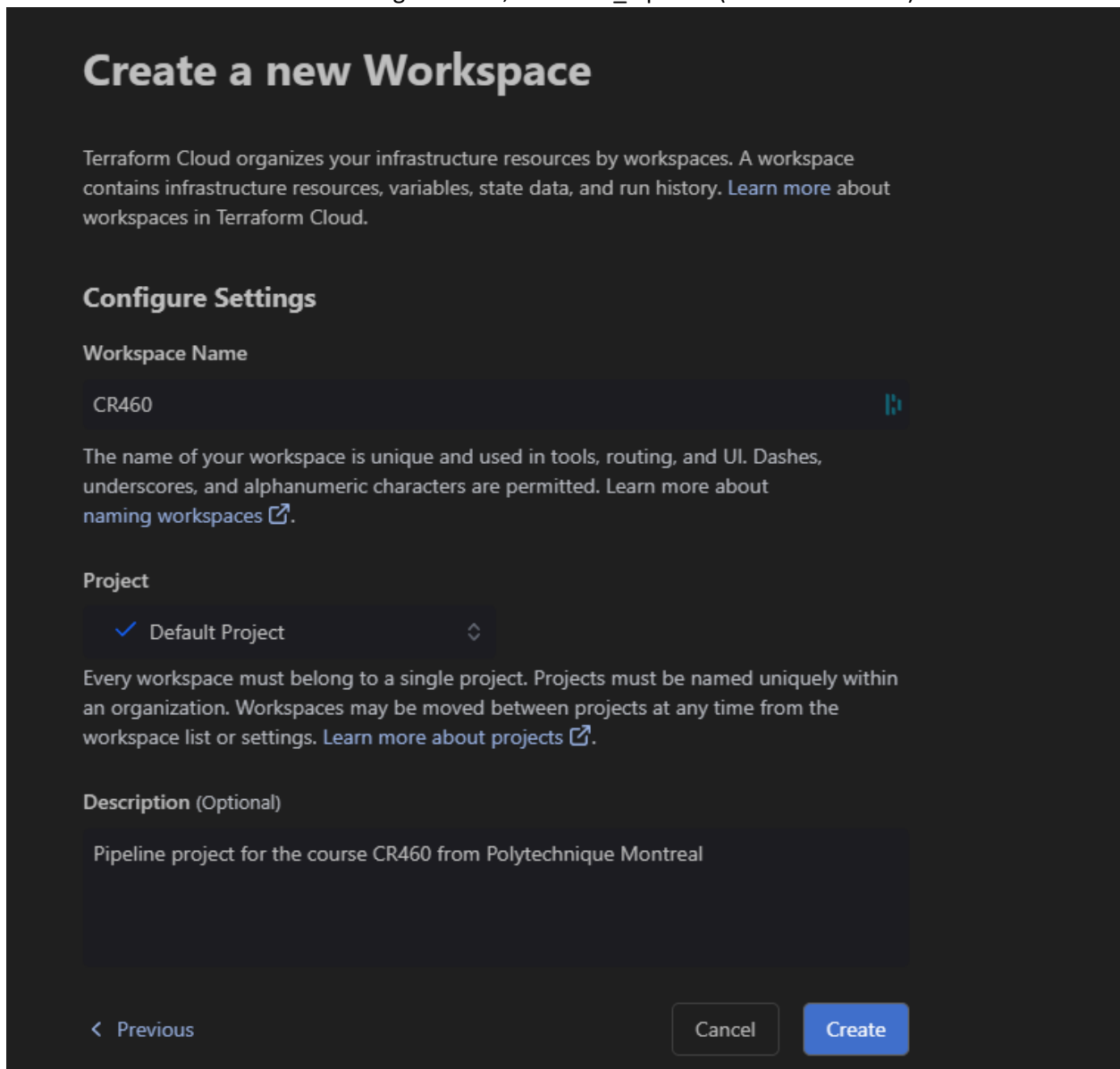
Please review the [Terms of Use](#) and [Privacy Policy](#).

Create account

We choose this option



I chose the name of the course as organization, so CR460_Pipeline (CR460 was taken)



We choose the second option to create a workspace (VCS)

CR460_Pipeline / Projects & workspaces / New Workspace

Create a new Workspace

Terraform Cloud organizes your infrastructure resources by workspaces. A workspace contains infrastructure resources, variables, state data, and run history. [Learn more about workspaces in Terraform Cloud.](#)

1 Connect to VCS

2 Choose a repository

3 Configure settings

Connect to a version control provider

Choose the version control provider that hosts the Terraform configuration for this workspace.

 GitHub ^

 GitLab v

 Bitbucket v

 Azure DevOps v

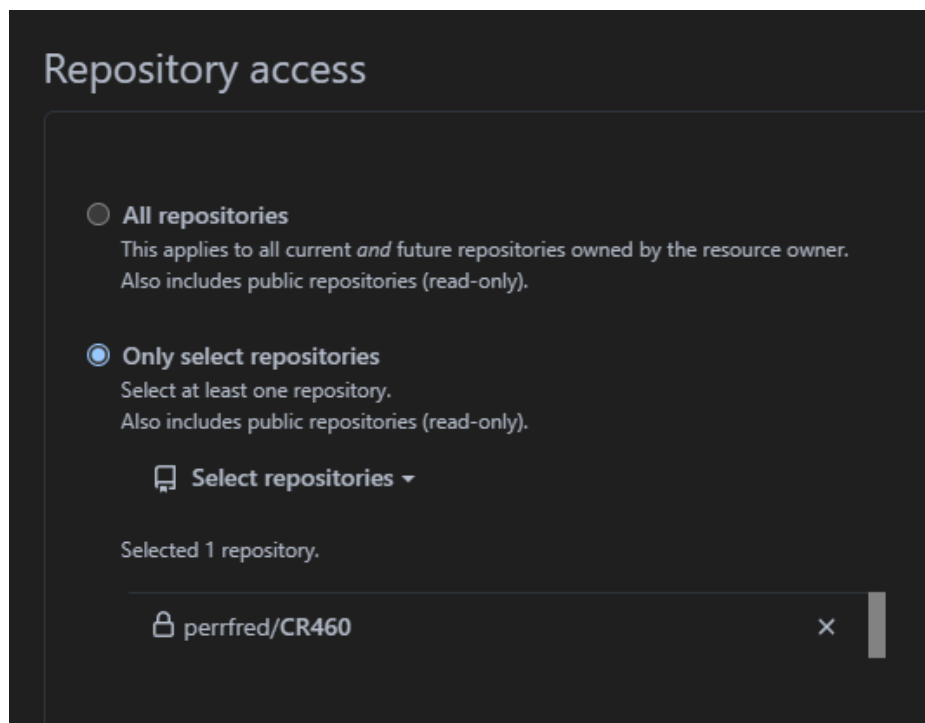
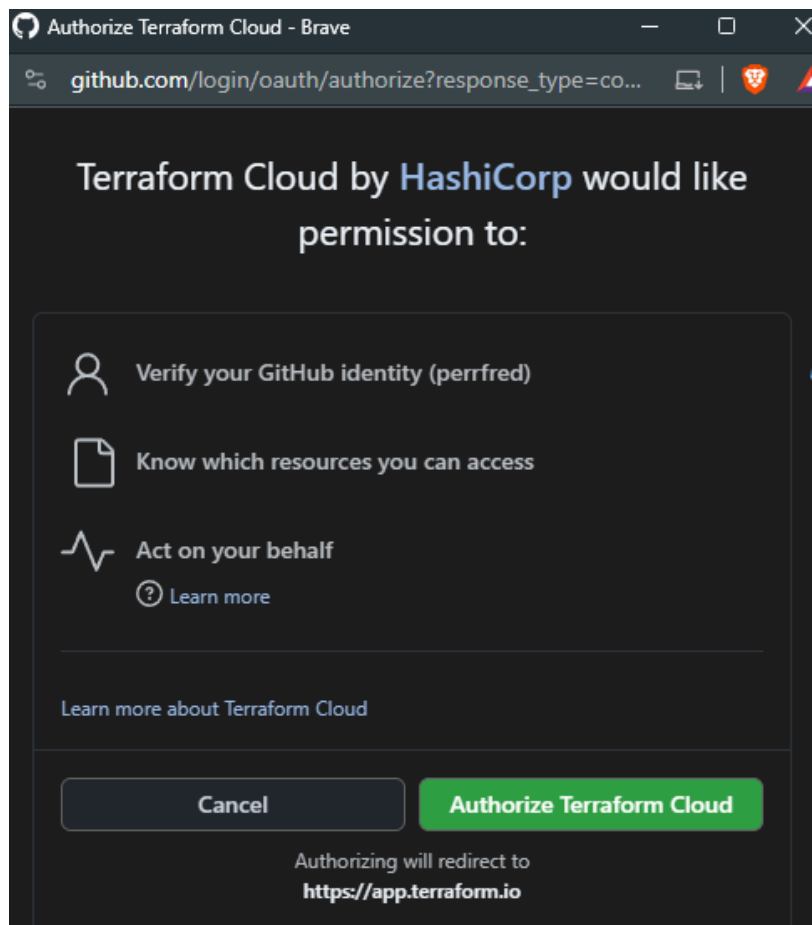
Version

GitHub.com

GitHub Enterprise

GitHub.com (Custom)

Cancel



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?

[Import a repository.](#)

Required fields are marked with an asterisk ().*

Owner *

 perrfred ▾

Repository name *

/ CR460

🔒 CR460 is available.

Great repository names are short and memorable. Need inspiration? How about **sturdy-system** ?

Description (optional)

- ☐  **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☒  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

☐ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: Terraform ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

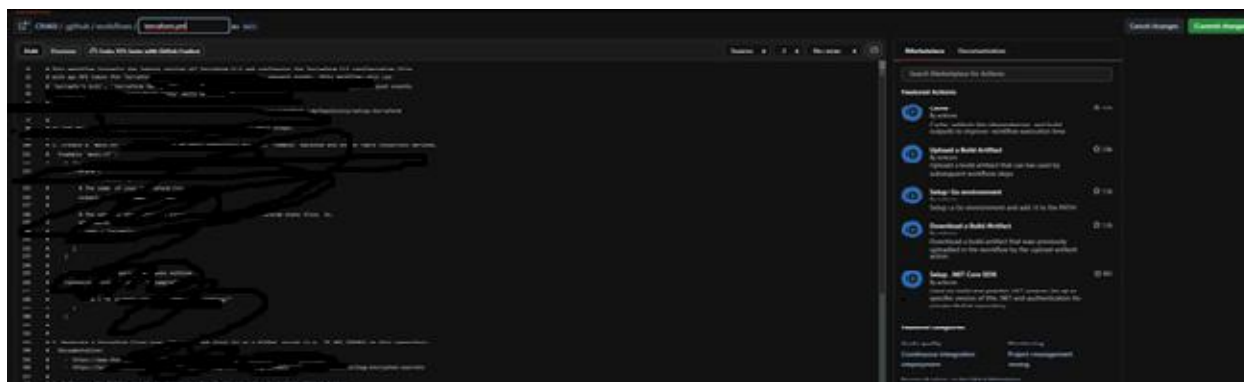
License: None ▾

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

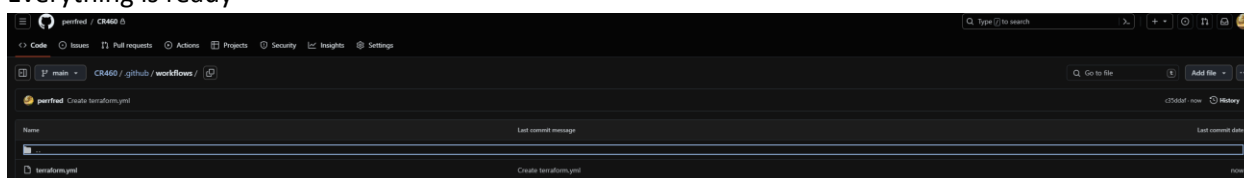
This will set  main as the default branch. Change the default name in your settings.

 You are creating a private repository in your personal account.

Create repository

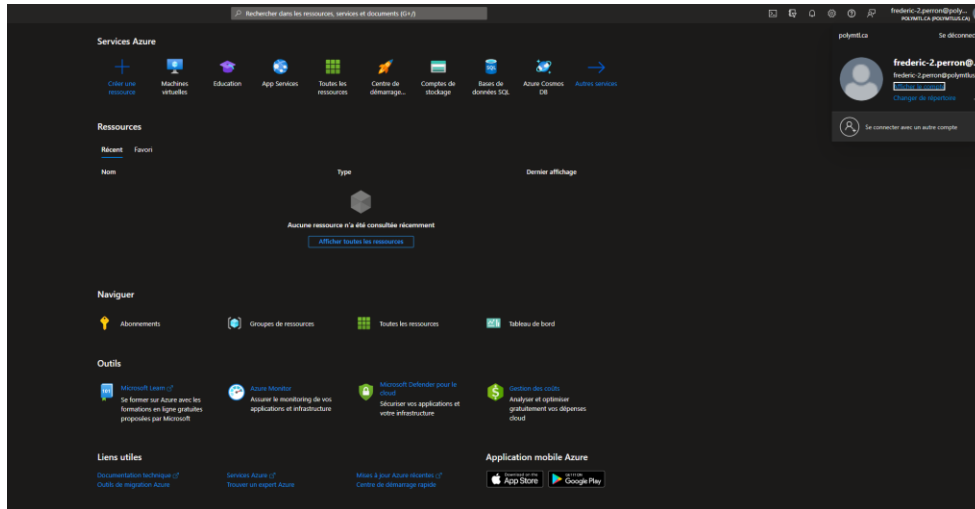


Everything is ready



Part 4 Configuration of Microsoft Azure account

I created an account using my Polytechnique email since it provides access to free services and other benefits. Here is the home page after creating and configuring my Microsoft Azure account



I then downloaded Azure CLI to access everything from my terminal

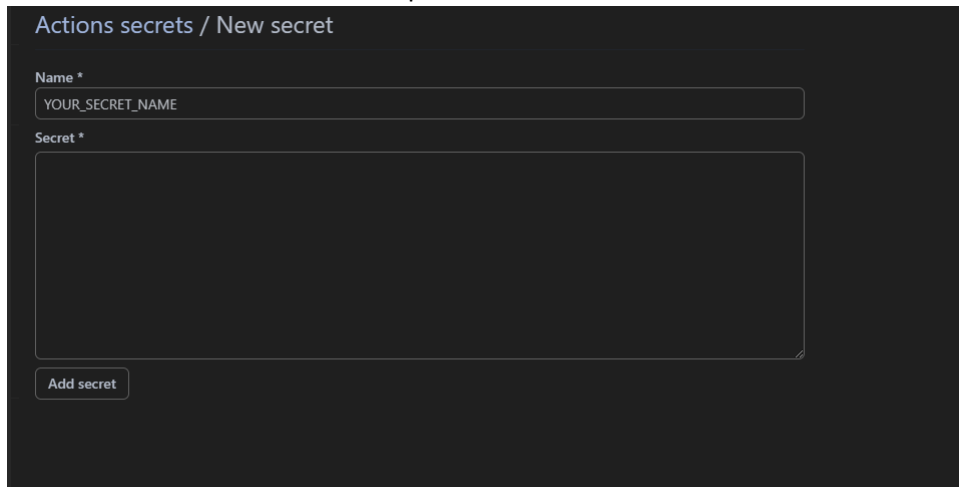
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\frede> az login
A web browser has been opened at https://login.microsoftonline.com/organizations/oauth2/v2.0/authorize. Please continue
the login in the web browser. If no web browser is available or if the web browser fails to open, use device code flow w
ith 'az login --use-device-code'.
[
  {
    "cloudName": "AzureCloud",
    "homeTenantId": "72f988bf-86f1-41af-91ab-2d7cd011db47",
    "id": "72f988bf-86f1-41af-91ab-2d7cd011db47",
    "isDefault": true,
    "managedByTenants": [],
    "name": "Azure for Students",
    "state": "Enabled",
    "tenantId": "72f988bf-86f1-41af-91ab-2d7cd011db47",
    "user": {
      "name": "frederic-2.perron@polymtlus.ca",
      "type": "user"
    }
  }
]
PS C:\Users\frede> |
```

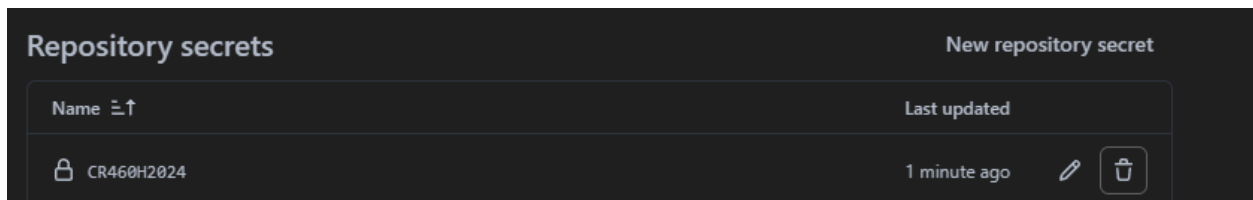
Part 5 Integration and connection between Github and Terraform

We create an API token and then proceed to create an **Actions secret**.



The screenshot shows the 'Actions secrets / New secret' interface. It has a dark theme. At the top, it says 'Actions secrets / New secret'. Below that, there are two main fields: 'Name *' and 'Secret *'. The 'Name *' field contains the placeholder text 'YOUR_SECRET_NAME'. The 'Secret *' field is a large text area. At the bottom left, there is a button labeled 'Add secret'.

We name it **CR460H2024**, consistent with our other repositories, and paste the key into the **“Secret”** field. We then establish the connection between Terraform and GitHub. *See image below*

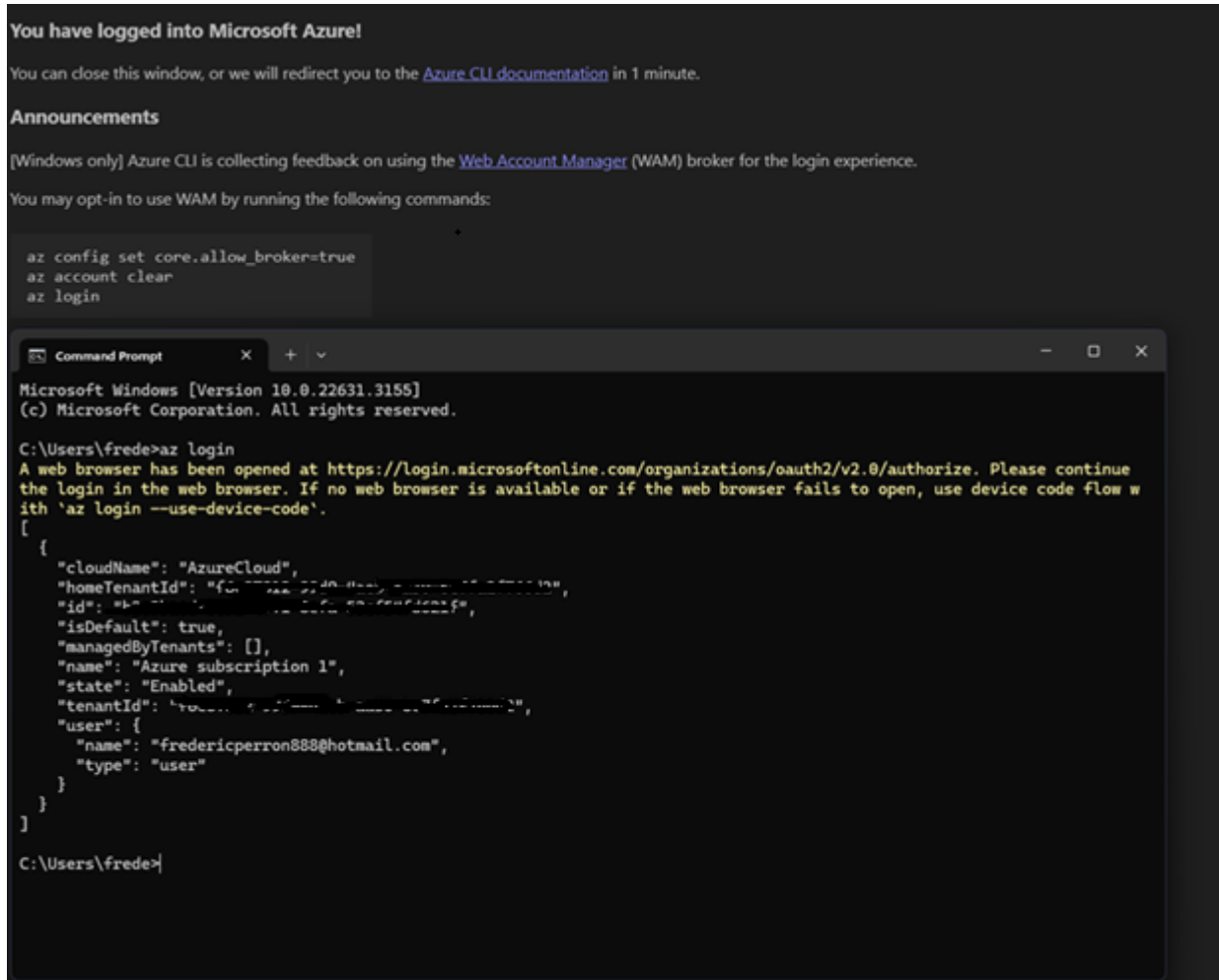


The screenshot shows the 'Repository secrets' page in GitHub. The title 'Repository secrets' is on the left, and 'New repository secret' is on the right. Below the title is a table with two columns: 'Name' and 'Last updated'. The 'Name' column has a sort icon (three horizontal lines and an upward arrow). The table contains one row with the name 'CR460H2024' and the last updated time '1 minute ago'. To the right of the '1 minute ago' text are two icons: a pencil (edit) and a trash can (delete).

Name 	Last updated
 CR460H2024	1 minute ago  

Part 6 Integration and connection between Terraform cloud & MS Azure

We start by connecting with Azure CLI from our Command Prompt



The screenshot shows the Azure CLI login process. At the top, a message states "You have logged into Microsoft Azure!" and provides a link to the Azure CLI documentation. Below this, an "Announcements" section mentions the Web Account Manager (WAM) broker. A code block lists the commands: `az config set core.allow_broker=true`, `az account clear`, and `az login`. The main part of the image is a Windows Command Prompt window titled "Command Prompt" showing the execution of `az login`. It displays the Microsoft Windows version (10.0.22631.3155) and the user's path (C:\Users\frede). The output of the `az login` command shows a JSON object with details about the Azure subscription and the user.

```
You have logged into Microsoft Azure!

You can close this window, or we will redirect you to the Azure CLI documentation in 1 minute.

Announcements

[Windows only] Azure CLI is collecting feedback on using the Web Account Manager (WAM) broker for the login experience.
You may opt-in to use WAM by running the following commands:

az config set core.allow_broker=true
az account clear
az login

C:\Users\frede>az login
A web browser has been opened at https://login.microsoftonline.com/organizations/oauth2/v2.0/authorize. Please continue
the login in the web browser. If no web browser is available or if the web browser fails to open, use device code flow w
ith 'az login --use-device-code'.
[
  {
    "cloudName": "AzureCloud",
    "homeTenantId": "f0123456-7890-1234-5678-901234567890",
    "id": "12345678-9012-3456-7890-123456789012",
    "isDefault": true,
    "managedByTenants": [],
    "name": "Azure subscription 1",
    "state": "Enabled",
    "tenantId": "12345678-9012-3456-7890-123456789012",
    "user": {
      "name": "fredericperron888@hotmail.com",
      "type": "user"
    }
  }
]
C:\Users\frede>
```

We can see when we do *init*, that everything is connected together as it should.

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

- Finding hashicorp/azurerm versions matching "3.93.0"...
- Installing hashicorp/azurerm v3.93.0...
- Installed hashicorp/azurerm v3.93.0 (signed by HashiCorp)

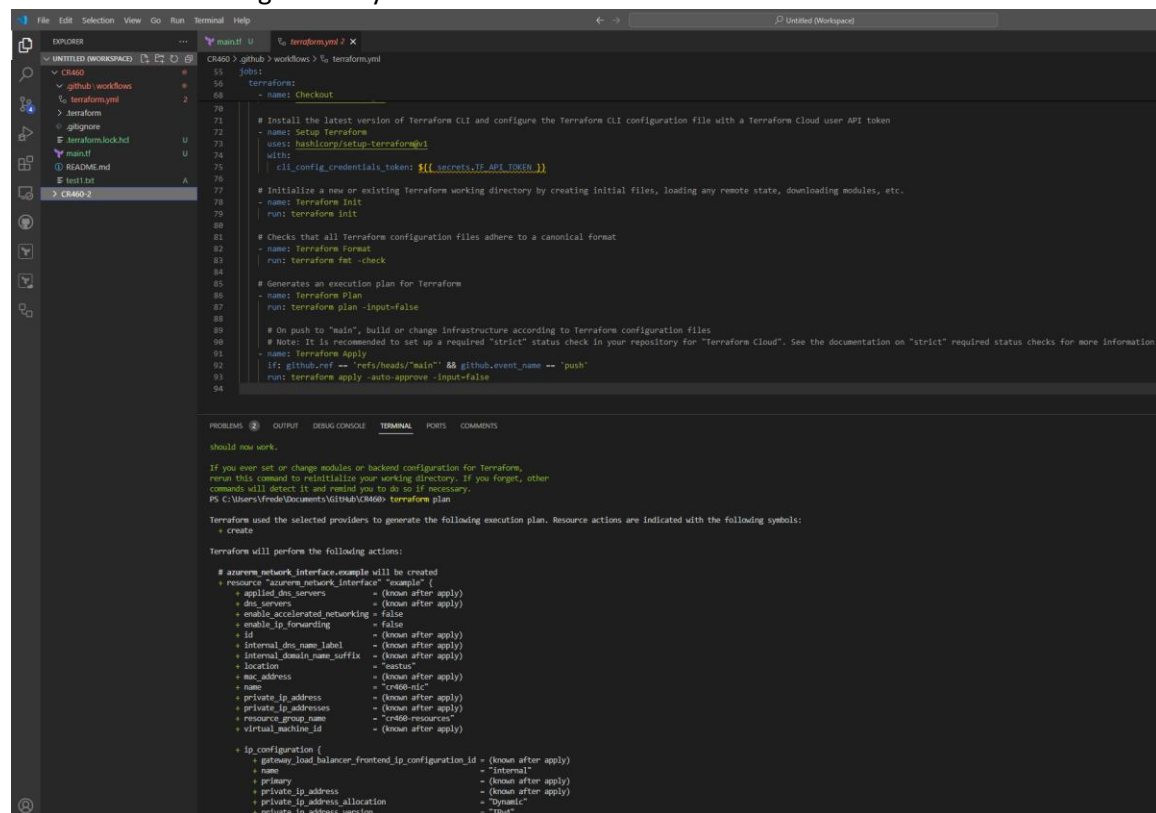
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\frede\Documents\GitHub\CR460>
```

Everything deploys as expected, demonstrating that the integration is properly configured and that the connection is working correctly.



The screenshot shows a VS Code editor with a workflow file open in the editor pane. The workflow file is a YAML document with the following content:

```
name: CR460
on:
  push:
    branches:
      - main
jobs:
  terraform:
    name: Checkout
    runs-on: ubuntu-latest
    steps:
      - name: Install the latest version of Terraform CLI and configure the Terraform CLI configuration file with a Terraform Cloud user API token
        uses: hashicorp/setup-terraform@v1
        with:
          cli_config_credentials_token: ${{ secrets.TF_API_TOKEN }}
      - name: Initialize a new or existing Terraform working directory by creating initial files, loading any remote state, downloading modules, etc.
        run: terraform init
      - name: Checks that all Terraform configuration files adhere to a canonical format
        run: terraform fmt -check
      - name: Generates an execution plan for Terraform
        run: terraform plan -input=false
      - name: On push to "main", build or change infrastructure according to Terraform configuration files
        # Note: It is recommended to set up a required "strict" status check in your repository for "Terraform Cloud". See the documentation on "strict" required status checks for more information.
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: terraform apply -auto-approve -input=false
```

The terminal pane shows the output of the workflow run. It starts with a message: "should now work." followed by a reminder to rerun the command if modules or backend configuration change. Then, it shows the command "PS C:\Users\frede\Documents\GitHub\CR460> terraform plan" and the resulting execution plan. The plan shows that the following actions will be performed:

- create
- azurerm_network_interface-example will be created
- resource "azurerm_network_interface" "example" {
- applied_dns_servers = (known after apply)
- dns_servers = (known after apply)
- enable_accelerated_networking = false
- enable_ip_forwarding = false
- id = (known after apply)
- internal_dns_name_label = (known after apply)
- internal_dns_name_suffix = (known after apply)
- location = "eastus"
- mac_address = (known after apply)
- name = "CR460-nic"
- private_ip_address = (known after apply)
- private_ip_addresses = (known after apply)
- resource_group_name = "CR460-resources"
- virtual_machine_id = (known after apply)
- }
- ip_configuration {
- gateway_load_balancer_frontend_ip_configuration_id = (known after apply)
- name = "internal"
- primary = (known after apply)
- private_ip_address = (known after apply)
- private_ip_address_allocation = "Dynamic"
- private_ip_address_version = "IPv4"
- }

Part 7 Deployment of group resources from your pipeline in MS Azure

We can see here the part of the script defining the resource groups, which will be deployed along with the other resources when I run terraform *init*, *plan*, and *apply* using Terraform.

```
9
10
11 provider "azurerm" {
12   features {}
13 }
14
15 resource "azurerm_resource_group" "example" {
16   name     = "cr460-resources"
17   location = "East US"
18 }
```

The screenshot shows the Microsoft Azure portal interface for the 'cr460-resources' resource group. The 'Resources' tab is active, displaying a table of resources. The 'cr460-vm' resource is highlighted with a red circle.

Name	Type	Location
cr460-machine	Virtual machine	East US
cr460-machine_OSDisk_1_79cdeb36417d4f41866e9bd2ace75d8	Disk	East US
cr460-network	Virtual network	East US
cr460-vm	Network interface	East US

Part 8 Deployment of a virtual network from your pipeline in MS Azure

We can see here the part of the script defining the virtual network, which will be deployed along with the other resources when I run terraform *init*, *plan*, and *apply* using Terraform.

```
20 resource "azurerm_virtual_network" "example" {
21     name                = "cr460-network"
22     address_space       = ["10.0.0.0/16"]
23     location             = azurerm_resource_group.example.location
24     resource_group_name = azurerm_resource_group.example.name
25 }
26
```

Filter for any field... Type equals all Location equals all Add filter

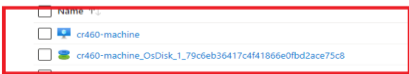
Showing 1 to 4 of 4 records. Show hidden types No grouping

Name ↑	Type ↑	Location ↑
cr460-machine	Virtual machine	East US
cr460-machine-OsDisk_1-79c6eb36417c4f41866e0fbd2ace75c8	Disk	East US
cr460-network	Virtual network	East US
cr460-nic	Network interface	East US

Part 9 Deployment of a VM from your pipeline in MS Azure

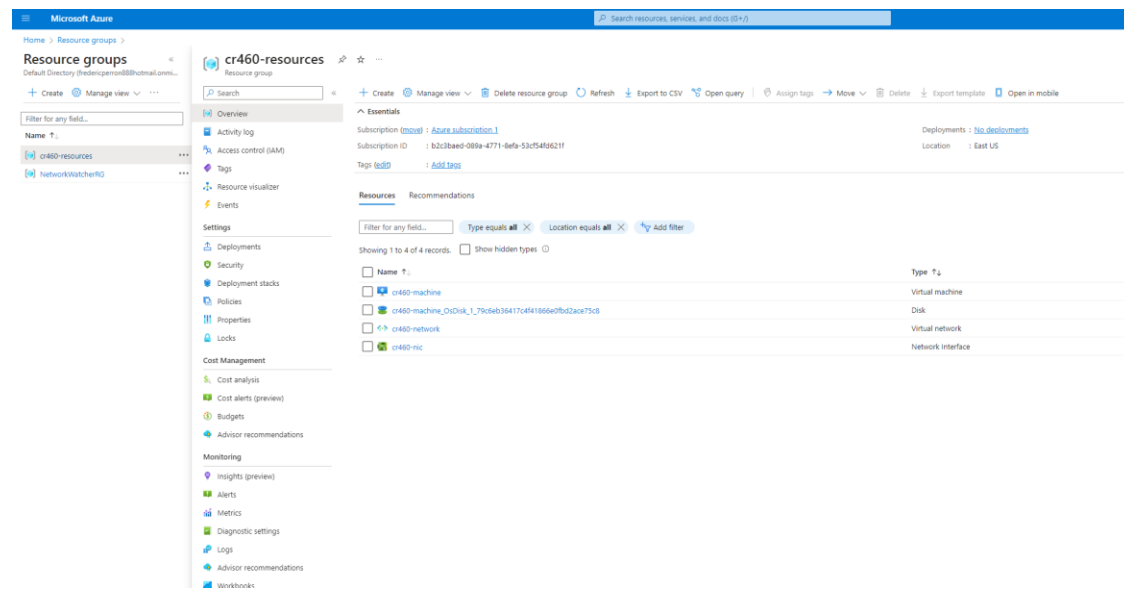
We can see here the part of the script defining the virtual machine, which will be deployed along with the other resources when I run terraform *init*, *plan*, and *apply* using Terraform.

```
46 resource "azurerm_windows_virtual_machine" "example" {
47   name                        = "cr460-machine"
48   resource_group_name        = azurerm_resource_group.example.name
49   location                   = azurerm_resource_group.example.location
50   size                       = "Standard_F2"
51   admin_username             = "adminuser"
52   admin_password             = "P@$$w0rd1234!"
53   network_interface_ids = [
54     azurerm_network_interface.example.id,
55   ]
56 }
```



Name	Type
cr460-machine	Virtual machine
cr460-machine_OsDisk_1_79c6eb36417c4f1866e0fbd2ace75cd	Disk
cr460-network	Virtual network
cr460-nic	Network interface

Après avoir fait *apply*, nous avons tous nos ressources disponibles dans Azure



Name	Type
cr460-machine	Virtual machine
cr460-machine_OsDisk_1_79c6eb36417c4f1866e0fbd2ace75cd	Disk
cr460-network	Virtual network
cr460-nic	Network interface

Microsoft Azure

Search resources, services, and docs (G+)

Home >

Resource groups

Default Directory (fdesigpenon88@hotmail.com@microsoft.com)

Create

Manage view

Refresh

Export to CSV

Open query

Assign tags

Filter for any field...

Subscription equals all

Location equals all

Add filter

Showing 1 to 2 of 2 records.

☐ Name ↑	Subscription ↑	Location ↑
☒ cr460-resources	Azure subscription 1	East US
☒ NetworkWatcherRG	Azure subscription 1	East US

Finally, we do *Destroy* to destroy our resources.

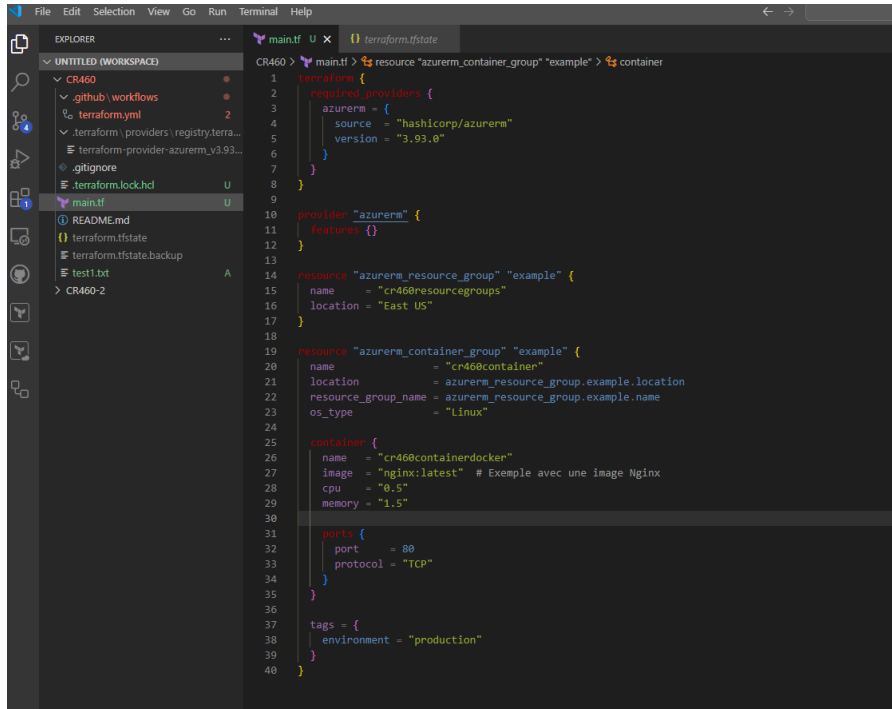
```

main.tf | U | % terraform.yml
CR460 > github > workflows > % terraform.yml
55 jobs:
56   terraform:
57     - name: Checkout
58       run: |
59         # Install the latest version of Terraform CLI and configure the Terraform CLI configuration file with a Terraform Cloud user API token
60         name: Setup Terraform
61         uses: hashicorp/setup-terraform@v1
62         with:
63           cli_config_credentials_token: ${{ secrets.TF_API_TOKEN }}
64           cli_config_credentials_token: ${{ secrets.TF_API_TOKEN }}
65
66         # Initialize a new or existing Terraform working directory by creating initial files, loading any remote state, downloading modules, etc.
67         - name: Terraform Init
68           run: terraform init
69
70         # Checks that all Terraform configuration files adhere to a canonical format
71         - name: Terraform Format
72           run: terraform fmt -check
73
74         # Generates an execution plan for Terraform
75         - name: Terraform Plan
76           run: terraform plan -input=false
77
78         # On push to "main", build or change infrastructure according to Terraform configuration files
79         # Note: It is recommended to set up a required "strict" status check in your repository for "Terraform Cloud". See the documentation on "strict" required status checks for more information: https://help.github.com/en/github/administering-a-repository/defining-the-strict-required-status
80         - name: Terraform Apply
81           if: ${{ github.ref == 'refs/heads/main' }}
82           run: terraform apply -auto-approve -input=false
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

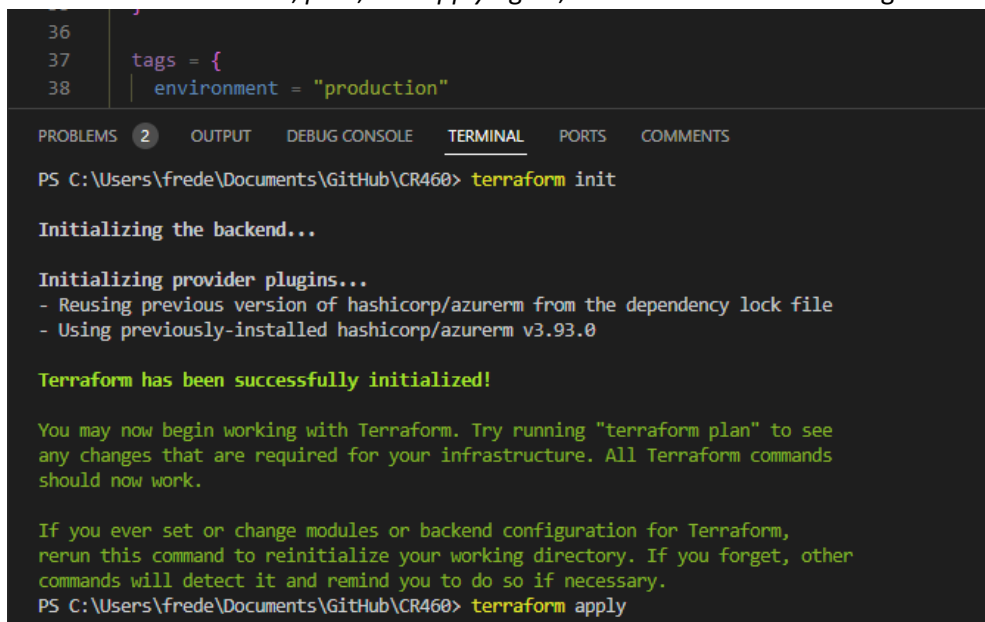
Part 10 Deployment of a Docker container from your pipeline in MS Azure

I replaced the sections for the other resources (VM, etc.) in my script with the following script to deploy my Docker container in Microsoft Azure:



```
1 terraform {
2   required_providers {
3     azure = {
4       source = "hashicorp/azure"
5       version = "3.93.0"
6     }
7   }
8 }
9
10 provider "azure" {
11   features {}
12 }
13
14 resource "azurerm_resource_group" "example" {
15   name     = "cr460resourcegroups"
16   location = "East US"
17 }
18
19 resource "azurerm_container_group" "example" {
20   name                = "cr460container"
21   location             = azurerm_resource_group.example.location
22   resource_group_name = azurerm_resource_group.example.name
23   os_type              = "Linux"
24
25   container {
26     name = "cr460containerdocker"
27     image = "nginx:latest" # Exemple avec une image Nginx
28     cpu   = "0.5"
29     memory = "1.5"
30
31     ports {
32       port     = 80
33       protocol = "TCP"
34     }
35   }
36
37   tags = {
38     environment = "production"
39   }
40 }
```

I then ran `terraform init`, `plan`, and `apply` again, and obtained the following results:



```
36
37 tags = {
38   environment = "production"
39 }
40 }
41 }
42 }
43 }
44 }
45 }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE **TERMINAL** PORTS COMMENTS

PS C:\Users\Frede\Documents\GitHub\CR460> terraform init

Initializing the backend...

Initializing provider plugins...

- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/azurerm v3.93.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

PS C:\Users\Frede\Documents\GitHub\CR460> terraform apply


```
main.tf U X terraform.tfstate
CR460 > main.tf > resource "azurerm_container_group" "example" > container > memory

1 terraform {
2   required_providers {
3     azurerm = {
4       source = "hashicorp/azurerm"
5       version = "3.93.0"
6     }
7   }
8 }
9
10 provider "azurerm" {
11   features {}
12 }
13
14 resource "azurerm_resource_group" "example" {
15   name     = "cr460resourcegroups"
16   location = "East US"
17 }
18
19
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\Frede\Documents\GitHub\CR460> terraform apply
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
# azurerm_container_group.example will be created
+ resource "azurerm_container_group" "example" {
  + dns_name_label_reuse_policy = "Unsecure"
  + exposed_port                = (known after apply)
  + fqdn                       = (known after apply)
  + id                         = (known after apply)
  + ip_address                 = (known after apply)
  + ip_address_type            = "Public"
  + location                   = "eastus"
  + name                       = "cr460container"
  + network_profile_id         = (known after apply)
  + os_type                    = "Linux"
  + resource_group_name        = "cr460resourcegroups"
  + restart_policy              = "Always"
  + sku                        = "Standard"
  + tags                       = {
    + "environment" = "production"
  }
  + container {
    + commands = (known after apply)
    + cpu      = 0.5
    + image    = "nginx:latest"
    + memory   = 1.5
    + name     = "cr460containerdocker"
    + ports {
      + port      = 80
      + protocol = "TCP"
    }
  }
}
# azurerm_resource_group.example will be created
+ resource "azurerm_resource_group" "example" {
  + id       = (known after apply)
  + location = "eastus"
  + name     = "cr460resourcegroups"
}
Plan: 2 to add, 0 to change, 0 to destroy.
```

```
main.tf | terraform.tfstate
CR460 > main.tf | terraform init
1 resource "azurerm_container_group" "example" {
2   container {
3     image = "mcr.microsoft.com/azurerm-test:latest"
4     name = "cr460container"
5     ports {
6       port = 80
7       protocol = "TCP"
8     }
9   }
10 }
11 provider "azurerm" {
12   version = "3.93.0"
13 }
14 resource "azurerm_resource_group" "example" {
15   name = "cr460resourcegroups"
16   location = "East US"
17 }
18 }
19 }
20 }
21 }
22 }
23 }
24 }
25 }
26 }
27 }
28 }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
```

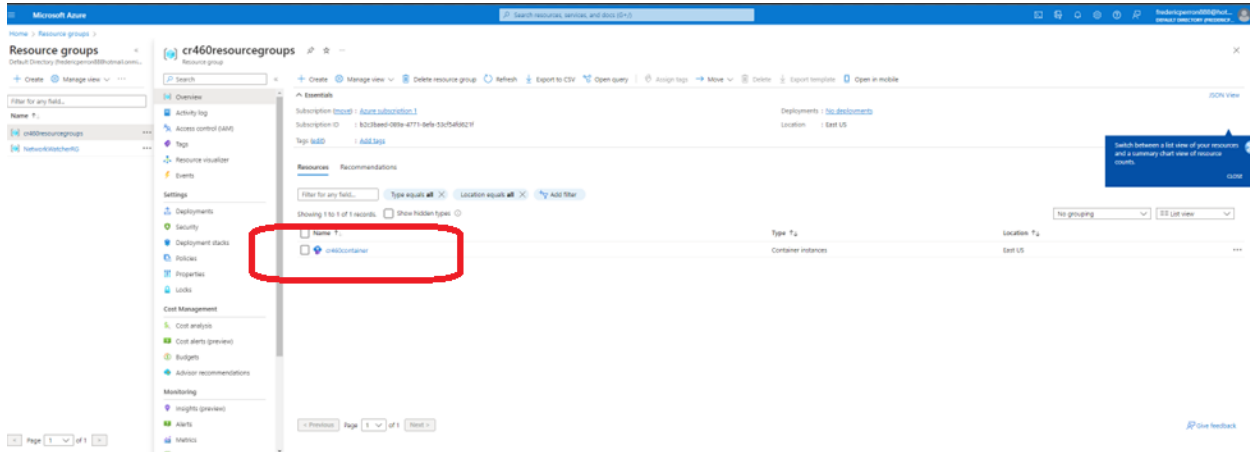
Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

azurerm_resource_group.example: Creating...
azurerm_resource_group.example: Creation complete after 1s [id=/subscriptions/b2c3baed-089a-4771-befa-53c54f6b21f/resourceGroups/cr460resourcegroups]
azurerm_container_group.example: Creating...
azurerm_container_group.example: Still creating... [10s elapsed]
azurerm_container_group.example: Still creating... [20s elapsed]
azurerm_container_group.example: Still creating... [30s elapsed]
azurerm_container_group.example: Still creating... [40s elapsed]
azurerm_container_group.example: Still creating... [50s elapsed]
azurerm_container_group.example: Still creating... [60s elapsed]
azurerm_container_group.example: Still creating... [70s elapsed]
azurerm_container_group.example: Still creating... [80s elapsed]
azurerm_container_group.example: Still creating... [90s elapsed]
azurerm_container_group.example: Creation complete after 1m3s [id=/subscriptions/b2c3baed-089a-4771-befa-53c54f6b21f/resourceGroups/cr460resourcegroups/providers/Microsoft.ContainerInstance/containerGroups/cr460container]
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

We can then see in the Microsoft Azure interface that our container named “cr460container” has been successfully created.



We then perform *terraform Destroy* to make sure everything's well removed.

```
PS C:\Users\fred\Documents\GitHub\CR460> terraform destroy
azurem_resource_group.example: Refreshing state... [id=/subscriptions/b2c3baed-089a-4771-8efa-53cf54fd621f/resourceGroups/cr460resourcegroups]
azurem_container_group.example: Refreshing state... [id=/subscriptions/b2c3baed-089a-4771-8efa-53cf54fd621f/resourceGroups/cr460resourcegroups/providers/Microsoft.ContainerInstance/containerGroups/cr460container]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# azurem_container_group.example will be destroyed
- resource "azurem_container_group" "example" {
  - dns_name_label_reuse_policy = "Unsecure" -> null
}
```

```
39 }
40 }
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
# azurem_resource_group.example will be destroyed
- resource "azurem_resource_group" "example" {
  - id = "/subscriptions/b2c3baed-089a-4771-8efa-53cf54fd621f/resourceGroups/cr460resourcegroups" -> null
  - location = "eastus" -> null
  - name = "cr460resourcegroups" -> null
  - tags = {} -> null
}
```

Plan: 0 to add, 0 to change, 2 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```
azurem_container_group.example: Destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-53cf54fd621f/resourceGroups/cr460resourcegroups/providers/Microsoft.ContainerInstance/containerGroups/cr460container]
azurem_container_group.example: Destruction complete after 2s
azurem_resource_group.example: Destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-53cf54fd621f/resourceGroups/cr460resourcegroups]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 10s elapsed]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 20s elapsed]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 30s elapsed]
```

```
39 }
40 }
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
- tags = {} -> null
}
```

Plan: 0 to add, 0 to change, 2 to destroy.

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

```
azurem_container_group.example: Destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-53cf54fd621f/resourceGroups/cr460resourcegroups/providers/Microsoft.ContainerInstance/containerGroups/cr460container]
azurem_container_group.example: Destruction complete after 2s
azurem_resource_group.example: Destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-53cf54fd621f/resourceGroups/cr460resourcegroups]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 10s elapsed]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 20s elapsed]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 30s elapsed]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 40s elapsed]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 50s elapsed]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 1m0s elapsed]
azurem_resource_group.example: Still destroying... [id=/subscriptions/b2c3baed-089a-4771-8efa-...21f/resourceGroups/cr460resourcegroups, 1m10s elapsed]
azurem_resource_group.example: Destruction complete after 1m17s
```

Destroy complete! Resources: 2 destroyed.
PS C:\Users\fred\Documents\GitHub\CR460>

Conclusion

In conclusion, this project provided valuable hands-on experience in installing, configuring, and integrating essential tools for cloud-based development and deployment. We successfully set up and configured Visual Studio Code, GitHub, Terraform CLI, and Terraform Cloud, and integrated them effectively with Microsoft Azure.

In addition, we deployed multiple Azure resources—including resource groups, virtual networks, virtual machines, and Docker containers—using a continuous deployment pipeline. This project strengthened our technical skills and improved our understanding of best practices in infrastructure management and cloud deployment.

References

- https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/windows_virtual_machine.html
- Courses Audio/Visual #2,3,4,5,6 CR460 and their respective PDFs